tests as examples when designing and defining your test scripts.

Now let’s run the test script, using Ganache and Truffle, and examine the output.

```
Contract: Ballot
  ✓ Success on initialization to registration phase. (55ms)
  Voter registration
    ✓ Success on registration of voters by chairperson.
    ✓ Failure on registration of voters by non-chairperson entity. (69ms)
    ✓ Failure on registration of voters in invalid phase. (281ms)
  Voting
    ✓ Success on vote. (189ms)
    ✓ Failure on voting for invalid candidate. (155ms)
    ✓ Failure on repeat vote. (195ms)
    ✓ Failure on vote by an unregistered user. (56ms)
    ✓ Failure on vote in invalid phase. (184ms)
  Phase Change
    ✓ Success on phase increment (99ms)
    ✓ Failure on phase decrement. (51ms)
    ✓ Failure on phase change by non-chairperson entity. (217ms)
  Requesting winner
    ✓ Success on query of winner with majority. (346ms)
    ✓ Success on query for the winner by a non-chairperson entity. (357ms)
    ✓ Success on tie-breaker when multiple candidates tied for the majority. (595ms)
    ✓ Failure on request for winner with majority vote less than three. (281ms)
    ✓ Failure on request for winner in invalid phase. (126ms)


17 passing (6s)
```

Figure 10.3 Test outputs for the ballot smart contract

10.3.2 Executing the ballot test script

Download the code ballot-contract from the codebase of the chapter. Start the Ganache test chain by clicking its icon and then clicking Quickstart. Then navigate to the ballot-contract directory. Install the node modules required, using the `npm` command. Then run the test command, using `truffle test`. This command picks up the JS test file ballotTest.js